GUÍA DE LABORATORIO 07

Fundamentos y tipos de recursividad

Definición, casos base y recursivo, pila de llamadas, diseño de algoritmos y clasificación de la recursión en Java

RUTA DE LA GUÍA

01 Definición → 02 Casos → 03 Pila de llamadas → 04 Diseño → 05 Tipos

UNIVERSIDAD	Universidad Autónoma Tomás Frías — Facultad de Ciencias Puras
CARRERA	Ingeniería Informática
ASIGNATURA	INF 220 — Programación Orientada a Objetos y Estructuras de Datos
UNIDAD	Unidad 2 · Recursividad
DURACIÓN	4 horas académicas (2 sesiones de laboratorio)
ENTORNO	JDK Temurin 25 (LTS) · Visual Studio Code · Extension Pack for Java

NOTA · **Instrucciones generales**

- Lea el marco teórico antes de la sesión; las actividades guiadas asumen que conoce los conceptos.
- Cree el proyecto `g07-recursividad` y una clase por actividad, respetando los nombres indicados.
- Para cada método recursivo escriba en un comentario su **caso base**, su **caso recursivo** y el **tipo de recursión**.
- Entregue el proyecto comprimido (.zip) y un informe breve (PDF) con las trazas y tablas solicitadas.

Estudiante: Fecha:

01 OBJETIVOS

▪ Objetivo general

Comprender el mecanismo de la recursividad y aplicarlo en el diseño de métodos recursivos correctos en Java, identificando sus casos, su comportamiento en la pila de llamadas y el tipo de recursión que emplean.

▪ Objetivos específicos

1. Definir la recursividad y reconocer sus dos componentes: **caso base** y **caso recursivo**.
2. Trazar la ejecución de un método recursivo mostrando la evolución de la **pila de llamadas**.
3. Aplicar una receta de diseño en tres pasos para construir métodos recursivos que terminen.
4. Clasificar la recursión en directa/indirecta, lineal/múltiple, anidada y de cola.
5. Estimar de forma intuitiva el costo de un algoritmo recursivo contando sus llamadas.

02 REQUISITOS

Elemento	Detalle
Software	JDK **Temurin 25 (LTS)**, **Visual Studio Code** con **Extension Pack for Java**
Conocimientos previos	Guías G01–G06: sintaxis de Java, métodos estáticos, parámetros, `String` y colecciones
Verificación	Ejecutar `java -version` en la terminal y comprobar que la versión es 25
Estructura del proyecto	`g07-recursividad/src/` con una clase `.java` por actividad

03 MARCO TEÓRICO

▪ 3.1 Definición

Un método es **recursivo** cuando, para resolver un problema, se invoca a sí mismo (directa o indirectamente) sobre una versión **más pequeña** del mismo problema. La recursividad es la traducción a código de las definiciones inductivas de la matemática: se define la solución para el caso más simple y se expresa el caso general en función de uno más simple.

> DEFINICIÓN · **Factorial como definición recursiva**
>
> $n! = 1$, si $n \leq 1$ (caso base)
>
> $n! = n \cdot (n-1)!$, si $n > 1$ (caso recursivo)
>
> Observe que la definición usa el propio concepto (factorial) pero sobre un valor estrictamente menor, lo que garantiza que en algún momento se llega al caso base.

▪ 3.2 Casos de la recursividad

Todo método recursivo bien construido tiene al menos dos casos:

- **Caso base (o de parada):** se resuelve directamente, sin llamadas recursivas. Detiene la recursión. Puede haber más de uno.

- **Caso recursivo (o general):** el método se llama a sí mismo con argumentos que se **acercan** al caso base, y combina el resultado.
- **Condición de convergencia:** cada llamada debe reducir el tamaño del problema (n − 1, n / 2, subcadena más corta, índice que avanza, etc.). Si no hay convergencia, la recursión es infinita.

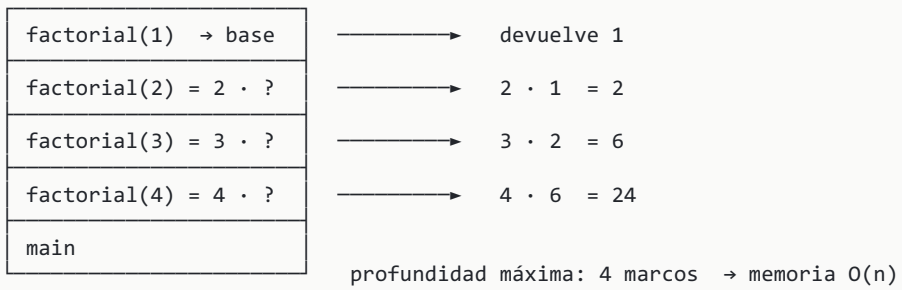
> **ADVERTENCIA · Recursión infinita**
> Olvidar el caso base, o escribir un caso recursivo que no se acerca a él (por ejemplo `factorial(n)` en lugar de `factorial(n - 1)`), provoca llamadas ilimitadas hasta agotar la memoria de la pila. En Java esto termina con `java.lang.StackOverflowError`.

3.3 La pila de llamadas

Cada vez que se invoca un método, la JVM crea un **marco de activación** (*stack frame*) que guarda sus parámetros, variables locales y el punto de retorno. Los marcos se apilan: el último en entrar es el primero en salir (LIFO). En un método recursivo coexisten tantos marcos como llamadas pendientes haya; la memoria usada es proporcional a la **profundidad** de la recursión.

La ejecución tiene dos fases: la **fase de ida** (las llamadas se apilan hasta alcanzar el caso base) y la **fase de vuelta** (los marcos se desapilan y cada uno completa su cálculo con el resultado recibido).

DIAGRAMA · PILA DE LLAMADAS DE factorial(4)



3.4 Diseño de algoritmos recursivos

Para construir un método recursivo se recomienda seguir siempre la misma receta:

Paso	Pregunta guía	Ejemplo: suma de dígitos de 4 725
1. Caso base	¿Cuál es la instancia más pequeña que puedo resolver sin recursión?	Si n < 10, la suma es n
2. Reducción	¿Cómo obtengo un problema del mismo tipo pero más pequeño?	Quitar el último dígito: n / 10 → 472
3. Combinación	Suponiendo que la llamada recursiva funciona, ¿cómo construyo la respuesta?	Último dígito + suma(472) → 5 + 13 = 18

> NOTA · **El "salto de fe" recursivo**
> En el paso 3 no hay que seguir mentalmente todas las llamadas: se **asume** que la llamada sobre el problema más pequeño devuelve el resultado correcto (hipótesis inductiva) y solo se razona cómo combinarlo. La traza completa sirve para depurar, no para diseñar.

● Plantilla.java Java

```java
static TipoResultado resolver(Problema p) {
    if (esCasoBase(p)) {                  // 1. caso base
        return solucionDirecta(p);
    }
    Problema menor = reducir(p);          // 2. reducción (debe converger)
    TipoResultado parcial = resolver(menor);
    return combinar(p, parcial);          // 3. combinación
}
```

▪ 3.5 Tipos de recursión

La recursión se clasifica según varios criterios. Un mismo método puede pertenecer a más de una categoría (por ejemplo, el factorial es directo, lineal y no de cola).

Tipo	Característica	Ejemplo	Costo típico
Directa	El método se invoca a sí mismo	`factorial(n - 1)`	—
Indirecta (mutua)	A llama a B y B llama a A	`esPar` ↔ `esImpar`	`O(n)`
Lineal (simple)	Cada invocación hace como máximo una llamada recursiva	factorial, potencia, MCD	`O(n)` típico
Múltiple (en árbol)	Cada invocación hace dos o más llamadas	Fibonacci ingenuo	$O(2^n)$ en el caso de Fibonacci
Anidada	Un argumento de la llamada es otra llamada recursiva	Función de Ackermann	Crecimiento explosivo
De cola	La llamada recursiva es la **última** operación; no queda nada pendiente al volver	factorial con acumulador	`O(n)` (Java no la optimiza)

> EJEMPLO · **Cómo reconocer la recursión de cola**
> `return n * factorial(n - 1);` **no** es de cola: al volver de la llamada todavía falta multiplicar por n.
> `return factorialCola(n - 1, acc * n);` **sí** es de cola: el resultado de la llamada se devuelve tal cual. El trabajo pendiente viaja en el parámetro acumulador.

04 ACTIVIDADES GUIADAS

A1 Factorial con traza de la pila

Implemente el factorial agregando un parámetro `nivel` que indique la profundidad de la llamada. La sangría permite **visualizar** la fase de ida y la de vuelta.

Factorial.java Java

```java
public class Factorial {

    // Caso base: n <= 1 | Caso recursivo: n * factorial(n - 1)
    // Tipo: directa, lineal, NO de cola
    static long factorial(int n, int nivel) {
        if (n < 0) {
            throw new IllegalArgumentException("n debe ser >= 0");
        }
        String sangria = "    ".repeat(nivel);
        System.out.println(sangria + "-> factorial(" + n + ")");
        if (n <= 1) {                                   // caso base
            System.out.println(sangria + "<- devuelve 1");
            return 1;
        }
        long resultado = n * factorial(n - 1, nivel + 1); // caso recursivo
        System.out.println(sangria + "<- devuelve " + resultado);
        return resultado;
    }

    public static void main(String[] args) {
        System.out.println("4! = " + factorial(4, 0));
    }
}
```

SALIDA ESPERADA

```
-> factorial(4)
    -> factorial(3)
        -> factorial(2)
            -> factorial(1)
            <- devuelve 1
        <- devuelve 2
    <- devuelve 6
<- devuelve 24
4! = 24
```

> **ADVERTENCIA · Desbordamiento de long**
> El tipo `long` solo almacena correctamente hasta 20! = 2 432 902 008 176 640 000. Pruebe `factorial(21, 0)` y observe el resultado incorrecto (negativo). Para valores mayores se usa `java.math.BigInteger`.

Tarea A1: ejecute el programa con n = 6 y dibuje en su informe la pila en el momento de máxima profundidad.

A2 Potencia: lineal vs. divide y vencerás

Dos formas recursivas de calcular b^e. La primera reduce el exponente en 1; la segunda lo **divide entre 2**, usando que $b^e = (b^{(e/2)})^2$ cuando e es par.

Potencia.java Java

```java
public class Potencia {

    static int llamadas = 0;

    // b^e = b * b^(e-1)        -> O(e) llamadas
    static long potencia(long b, int e) {
        llamadas++;
        if (e == 0) return 1;                  // caso base
        return b * potencia(b, e - 1);         // caso recursivo
    }

    // b^e = (b^(e/2))^2 [* b si e es impar]  -> O(log e) llamadas
    static long potenciaRapida(long b, int e) {
        llamadas++;
        if (e == 0) return 1;                  // caso base
        long mitad = potenciaRapida(b, e / 2); // UNA sola llamada, se reutiliza
        return (e % 2 == 0) ? mitad * mitad : mitad * mitad * b;
    }

    public static void main(String[] args) {
        int[] exponentes = {10, 20, 40, 60};
        for (int e : exponentes) {
            llamadas = 0;
            long r1 = potencia(2, e);
            int c1 = llamadas;
            llamadas = 0;
            long r2 = potenciaRapida(2, e);
            int c2 = llamadas;
            System.out.printf("2^%d = %d | lineal: %d llamadas | rapida: %d llamadas%n",
                    e, r1, c1, c2);
            assert r1 == r2;
        }
    }
}
```

> NOTA · **Detalle de diseño**
>
> Si en `potenciaRapida` se escribiera `potenciaRapida(b, e / 2) * potenciaRapida(b, e / 2)`, la recursión pasaría a ser **múltiple** y se perdería toda la ventaja: el número de llamadas volvería a ser `O(e)`. Guardar el resultado en `mitad` es lo que la hace `O(log e)`.

Tarea A2: complete la tabla con el número de llamadas obtenido.

Exponente e	Llamadas (lineal)	Llamadas (rápida)	¿Qué función describe cada columna?
10			
20			
40			
60			

A3 Suma de dígitos y máximo común divisor

Aplique la receta de diseño de la sección 3.4. El MCD usa el algoritmo de Euclides: mcd(a, b) = mcd(b, a mod b), con mcd(a, 0) = a.

Numeros.java Java

```java
public class Numeros {

    // Caso base: n < 10 | Reducción: n / 10 | Combinación: n % 10 + suma
    static int sumaDigitos(int n) {
        if (n < 10) return n;
        return n % 10 + sumaDigitos(n / 10);
    }

    // Caso base: b == 0 | Recursión de COLA (nada pendiente al volver)
    static int mcd(int a, int b) {
        if (b == 0) return a;
        return mcd(b, a % b);
    }

    public static void main(String[] args) {
        System.out.println("sumaDigitos(4725) = " + sumaDigitos(4725)); // 18
        System.out.println("mcd(252, 105)     = " + mcd(252, 105));     // 21
        System.out.println("mcd(17, 5)        = " + mcd(17, 5));        // 1
    }
}
```

Tarea A3: escriba la secuencia de llamadas de `mcd(252, 105)` hasta llegar al caso base. ¿Por qué la recursión converge aunque los argumentos no disminuyen de uno en uno?

A4 Palíndromo recursivo con índices

Una cadena es palíndromo si sus extremos coinciden y la subcadena interior también es palíndromo. En lugar de crear subcadenas con `substring` (que copia caracteres), se usan dos índices `i` y `j` que avanzan hacia el centro. Un método público "envoltorio" prepara la cadena y llama al método recursivo auxiliar.

Palindromo.java Java

```java
import java.text.Normalizer;

public class Palindromo {

    // Envoltorio: normaliza (quita tildes, espacios y signos) y delega
    static boolean esPalindromo(String s) {
        String limpio = Normalizer.normalize(s, Normalizer.Form.NFD)
                .replaceAll("\\p{M}", "")       // elimina marcas diacríticas
                .toLowerCase()
                .replaceAll("[^a-z0-9]", "");   // solo letras y dígitos
        return esPalindromo(limpio, 0, limpio.length() - 1);
    }

    // Casos base: i >= j (verdadero) o extremos distintos (falso)
    // Caso recursivo: interior de la cadena. Tipo: directa, lineal, de cola
    private static boolean esPalindromo(String s, int i, int j) {
        if (i >= j) return true;
        if (s.charAt(i) != s.charAt(j)) return false;
        return esPalindromo(s, i + 1, j - 1);
    }
```

```
    public static void main(String[] args) {
        String[] pruebas = {"Anita lava la tina", "Dábale arroz a la zorra el abad",
                            "Recursividad", "", "a"};
        for (String t : pruebas) {
            System.out.printf("%-35s -> %b%n", "\"" + t + "\"", esPalindromo(t));
        }
    }
}
```

EJEMPLO · Patrón envoltorio + auxiliar

Es muy frecuente que la versión recursiva necesite parámetros extra (índices, acumuladores, nivel) que el usuario del método no debería proporcionar. Se resuelve con un método público sencillo que llama a un método `private` recursivo con los parámetros adicionales. Este patrón se reutilizará en la G08 con listas.

A5 Un ejemplo por cada tipo de recursión

Implemente la clase `TiposRecursion` y ejecute cada método. El contador `llamadas` de Fibonacci permite observar el crecimiento explosivo de la recursión múltiple.

TiposRecursion.java Java

```
public class TiposRecursion {

    // ---------- MÚLTIPLE (en árbol): dos llamadas por invocación
    static long llamadas = 0;
    static long fib(int n) {
        llamadas++;
        if (n < 2) return n;                    // casos base: fib(0)=0, fib(1)=1
        return fib(n - 1) + fib(n - 2);
    }

    // ---------- INDIRECTA (mutua): esPar -> esImpar -> esPar ...
    static boolean esPar(int n) {
        if (n == 0) return true;
        return esImpar(n - 1);
    }
    static boolean esImpar(int n) {
        if (n == 0) return false;
        return esPar(n - 1);
    }

    // ---------- ANIDADA: el argumento es otra llamada recursiva
    static long ackermann(long m, long n) {
        if (m == 0) return n + 1;
        if (n == 0) return ackermann(m - 1, 1);
        return ackermann(m - 1, ackermann(m, n - 1));
    }

    // ---------- DE COLA: el trabajo pendiente viaja en el acumulador
    static long factorialCola(int n, long acc) {
        if (n <= 1) return acc;
        return factorialCola(n - 1, acc * n);
    }
    static long factorialCola(int n) {          // envoltorio
        return factorialCola(n, 1);
    }
```

```
    public static void main(String[] args) {
        for (int n : new int[]{10, 20, 25, 30}) {
            llamadas = 0;
            long r = fib(n);
            System.out.printf("fib(%d) = %d  -> %,d llamadas%n", n, r, llamadas);
        }
        System.out.println("esPar(10) = " + esPar(10) + ", esImpar(7) = " + esImpar(7));
        System.out.println("ackermann(2, 3) = " + ackermann(2, 3));
        System.out.println("ackermann(3, 3) = " + ackermann(3, 3));
        System.out.println("factorialCola(10) = " + factorialCola(10));
    }
}
```

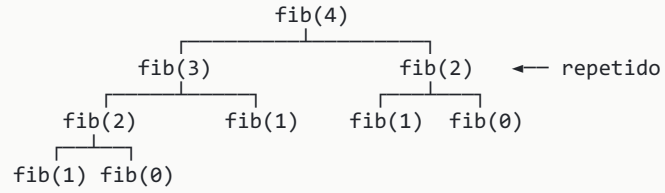
SALIDA ESPERADA

```
fib(10) = 55  -> 177 llamadas
fib(20) = 6765  -> 21,891 llamadas
fib(25) = 75025  -> 242,785 llamadas
fib(30) = 832040  -> 2,692,537 llamadas
esPar(10) = true, esImpar(7) = true
ackermann(2, 3) = 9
ackermann(3, 3) = 61
factorialCola(10) = 3628800
```

El separador de miles puede mostrarse como punto o coma según la configuración regional del equipo.

El árbol de llamadas de `fib(4)` muestra por qué la recursión múltiple es costosa: subproblemas como `fib(2)` se calculan **más de una vez**.

DIAGRAMA · ÁRBOL DE LLAMADAS DE fib(4)



9 llamadas para n = 4 · crecimiento ≈ O(ϕⁿ), ϕ ≈ 1.618

> **ADVERTENCIA · Ackermann crece de forma explosiva**
>
> No ejecute `ackermann(4, 1)` o valores mayores: la cantidad de llamadas y la profundidad de la pila son enormes y el programa terminará con `StackOverflowError` o no terminará en un tiempo razonable. Es un ejemplo teórico de función recursiva que no puede expresarse con recursión "primitiva".

Tarea A5: para cada método de la clase escriba en su informe su tipo de recursión según **todos** los criterios de la tabla 3.5 (por ejemplo: "directa, lineal, de cola").

05 BIBLIOGRAFÍA

- Joyanes Aguilar, L. y Zahonero Martínez, I. (2008). *Estructuras de datos en Java*. McGraw-Hill. Cap. 5: Algoritmos recursivos.
- Weiss, M. A. (2010). *Data Structures and Problem Solving Using Java* (4.ª ed.). Pearson. Cap. 7: Recursion.

- Deitel, P. y Deitel, H. (2017). *Java How to Program, Early Objects* (11.ª ed.). Pearson. Cap. 18: Recursion.
- Sedgewick, R. y Wayne, K. (2017). *Computer Science: An Interdisciplinary Approach*. Addison-Wesley. Sección 2.3: Recursion.
- Oracle. *The Java Tutorials* y documentación de la API de Java SE 25. https://docs.oracle.com/en/java/